Verifier-Assisted versus LLM-Only Pre-deployment Network-Change Validation: A Preregistered Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We measured whether integrating a deterministic pre-deployment verifier with a bounded automated repair budget increases the fraction of intent-driven network-change proposals that pass semantic validation compared with accepting a single LLM candidate. In a preregistered paired design (study id cnsmp2026-llm-verifier-predeployment-001) we executed 300 paired intents (100 per difficulty stratum: low/medium/high) using gpt-5-mini and deterministic_netops_validator_v5. Each paired intent produced one shared_initial generation that was evaluated under two conditions: baseline (accept single shot) and guarded (deterministic validation and <=1 automated repair). Outcomes: baseline 299/300 unflagged passes (0.9966667), guarded 300/300 (1.0); contingency n11=299, n10=1, n01=0, n00=0; absolute paired difference = 0.0033333; exact McNemar two-sided p = 1.0; 95% paired bootstrap percentile CI = [0.00,0.01] (bootstrap_seed = 1729, B = 10000). Under the supplied synthetic suite and repair budget the single-shot generator was near ceiling, limiting observable deterministic rescue. We archive preregistration, execution, and analysis artifacts and interpret results with respect to estimand conditioning, validator coverage, repair budget, and operational deployment policies.**

I. Introduction

Generative large language models (LLMs) are being applied to network operations (NetOps) tasks such as intent→configuration translation, configuration synthesis, and automated repair. For pre-deployment network changes, semantic correctness properties—sequencing constraints, reachability, and policy preservation—are operationally critical and cannot be assured by superficial syntactic checks alone. A common engineering pattern is to pair generative models with deterministic validators and bounded automated repair (generate→validate→repair) to reduce operational risk. However, the measurable benefit of such guarded pipelines depends on the experimental estimand (what is held fixed), the generator single-shot quality, the validator's expressiveness, and the permitted repair budget.

We preregistered a paired experiment (study id cnsmp2026-llm-verifier-predeployment-001) to quantify the deterministic rescue achievable when the same single LLM candidate is evaluated under two treatments. For each intent the pipeline produced exactly one shared_initial candidate using the declared generator (gpt-5-mini) and compared: (a) baseline — accept the single candidate without repair, and (b) guarded — run a deterministic validator (deterministic_netops_validator_v5) and allow at most one automated repair which itself is re-validated. The primary research question: does adding a deterministic verifier plus at most one automated repair increase the probability that a single LLM candidate passes semantic pre-deployment validation compared with accepting that same candidate without repair?

Contributions of this artifact-grounded study: (1) a preregistered paired evaluation isolating deterministic rescue under a shared_initial estimand that conditions on a single LLM draw per intent; (2) a complete execution and analysis bundle including validator traces and the single automated repair transcript archived with the run; and (3) an evidence-grounded interpretation of estimator conditioning, ceiling effects, and operational implications for NetOps pipelines.

II. Related Work

Recent surveys and systems work document active efforts to apply LLMs to NetOps tasks including intent translation, configuration synthesis, automated repair, and tool-augmented orchestration [https://openalex.org/W4402742293; https://openalex.org/W4411015066; https://openalex.org/W4416927413]. Architectures that interpose validators or tools (planner-centric frameworks, ReAct variants, and generate→validate→repair pipelines) are proposed to reduce hallucination and semantic errors; prior empirical gains vary substantially with validator coverage and generation strategy [doi:10.1609/aaai.v40i40.40676; https://openalex.org/W4404240614].

Two methodological gaps motivated the present design. First, many prior comparisons conflate deterministic validator rescue with benefits obtained by drawing multiple independent model candidates: permitting resampling mixes generator variance effects with deterministic validation effects. Second, community benchmarks emphasizing semantic correctness (reachability and policy compliance) rather than only syntactic validity are limited [https://openalex.org/W4415071524; https://openalex.org/W4392875514], making operationally meaningful comparison difficult. Our paired shared_initial design isolates deterministic rescue from resampling effects, providing an operational bound for policies that constrain model calls per change request.

III. Methodology

Preregistration and primary estimand. We preregistered the full confirmatory design under study id cnsmp2026-llm-verifier-predeployment-001 and archived the preregistration

SHA-256 = 425cb1652354120cf48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7. The preregistered primary estimand is the absolute paired difference in the unflagged verifier-pass proportion (guarded - baseline) computed on N = 300 paired intents after applying the prespecified inclusion/exclusion rules. The confirmatory analysis executor paired_binary_analysis_v1 and multiplicity handling (Benjamini–Hochberg for exploratory strata tests) were fixed before observing main-run outcomes. The preregistration described an optional calibration pilot (n = 60) for discordance estimation and power calibration; that pilot was not executed and no post-preregistration power recalculation was performed.

Task generation, stratification, and synthetic suite. Tasks were produced deterministically by the registered deterministic_netops_task_generator_v5 into a synthetic_topology_suite stratified into three difficulty strata (low/medium/high). Each task manifest entry encodes: (i) the natural-language intent, (ii) initial and expected final device states, (iii) required_sequence constraints (ordered field changes such as must_be_down_for_fields), (iv) preserve_unspecified_state invariants, and (v) protected_interfaces that must not be modified. Difficulty metadata (changed_interface_count, dependency_rule_count, required_command_count) was used to assign stratum membership and to ensure equal per-stratum sampling (100 pairs per stratum in the confirmatory run). Representative difficulty patterns appearing in the suite include 'shutdown_configure_restore' (medium) and 'paired_link_atomic_mtu_change' (hard).

Shared_initial generation protocol and model configuration. To isolate deterministic validator/repair rescue from generator resampling effects we implemented the shared_initial estimand: for each pair the pipeline produced exactly one shared_initial candidate which was reused verbatim for both baseline and guarded conditions. The declared generator for shared_initial production is gpt-5-mini as recorded in execution/model_configuration.json (requested_model = "gpt-5-mini", declared_model_version = "gpt-5-mini", max_output_tokens = 1500, reasoning_effort = "minimal"). Reusing the same candidate across conditions prevents differences due to independent additional model draws and measures only deterministic validation and bounded repair applied to the identical output.

Deterministic validator, scoring rule, and repair operator. Semantic evaluation used deterministic_netops_validator_v5 implementing the preregistered full_intent_constraint_validation rule. The validator deterministically checks sequencing invariants (e.g., must_be_down_for_fields), required_command_count, preservation of unspecified state, topology reachability invariants encoded by the synthetic suite, and forbidden-template protections for protected_interfaces. Scoring is binary: an unflagged candidate is considered a pass for the estimand. The guarded pipeline permits at most one automated repair attempt (repair budget <= 1). The repair operator is a constrained edit routine limited to localized sequencing and insertion edits (e.g., inserting a missing 'admin down' before a VLAN/MTU change) and is applied only if the validator flags the shared_initial candidate; repair attempts are logged with a repair_provider_trace and re-validated deterministically. Repair candidates that remain flagged after the permitted edit count as failures for the guarded condition in the primary analysis.

Contamination controls, negative controls, and exclusion/missingness rules. The preregistration included deterministic contamination detection: one intentionally broken negative control per complexity stratum and invariant checks comparing pre/post change reachability matrices. The inclusion/exclusion rules specified that a pair would be excluded from confirmatory analysis only if both pipeline calls for that pair failed due to provider/infrastructure outage; if only one call failed, the pair would be retained per the executor's complete_pair_primary policy with sensitivity analyses treating single failures as failures. The missingness plan required logging all provider/API failures; in the confirmatory run no dual-call outages occurred and no pairs were excluded.

Execution instrumentation, provider auditing, and determinism checks. The run used the hosted_netops_gvr_v1 adapter under execution_mode = scientific_confirmatory. Execution manifest values archived with the run include planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, and model_calls_used = 301 (hosted_model_call_event_count = 301). Provider auditing recorded cache_miss_count = 301 and stage_counts = {shared_initial_generation: 300, repair: 1}. Deterministic reconciliation verified episode accounting and pair consistency and reported all_deterministic_consistency_checks_passed = true. These instrumentation values and stage counts are archived to enable third-party verification of the execution integrity and of the shared_initial protocol (i.e., that the same shared_initial candidate was used across conditions).

Analysis executor, inferential procedures, and sensitivity analyses. The preregistered confirmatory analysis used paired_binary_analysis_v1 to compute contingency counts (n11,n10,n01,n00), the absolute paired difference (guarded - baseline), an exact two-sided McNemar test, and a paired bootstrap percentile 95% confidence interval. The bootstrap parameters used in the archived analysis are seed = 1729 and resamples B = 10000. The preregistration designated the primary test as the single confirmatory hypothesis (H1) evaluated on the main N = 300 pairs; secondary and stratum tests were labeled exploratory and subject to Benjamini–Hochberg FDR correction when reported. Preplanned sensitivity analyses included (a) failure-as-zero handling for single-call failures, (b) stratum-wise paired tests with multiplicity control, and (c) contamination checks; these were executed as exploratory artifacts and archived accordingly.

Stopping rule and operational constraints. The preregistered stopping rule required halting the confirmatory run if >20% of attempted pairs were excluded due to dual-call provider/infrastructure outages; this condition did not occur. The execution also respected the prespecified maximum_model_calls = 600 and the planned model-call budget; the run used model_calls_used = 301 consistent with the shared_initial protocol plus one repair invocation. All operationally relevant constraints (repair budget <= 1, single shared_initial generation per pair) were enforced by the adapter and logged in the execution manifest.

Reproducibility, archiving, and limits of the methodological conditioning. All design artifacts (preregistration, task manifests, model configuration, validator and repair traces, execution logs, and analysis inputs) are archived with the run bundle to permit deterministic replay of the confirmatory executor. Methodologically, the shared_initial estimand intentionally conditions on a single LLM draw per pair and thus excludes any verifier benefit that would arise from permitting independent alternative LLM candidates per condition (resampling or interactive verifier-guided regeneration). That distinction is central to interpreting the confirmatory null discordance observed on this suite and is explicitly encoded in the preregistration and archived executor configurations.

IV. Results

Execution integrity and accounting. The preregistered confirmatory run completed as planned with completed_episode_count = 600 (300 paired intents reused as shared_initial candidates) and complete_pair_count = 300. Provider auditing recorded hosted_model_call_event_count = 301 (300 shared_initial generations plus one repair invocation) and model_calls_used = 301; stage_counts indicate shared_initial_generation = 300 and repair = 1. Deterministic reconciliation confirms that all deterministic consistency checks passed (all_deterministic_consistency_checks_passed = true) and that no provider/infrastructure outages caused exclusions under the preregistered rules.

Primary confirmatory outcomes. Aggregate condition totals (analysis/condition_summary.csv) show baseline successes = 299/300 (0.9966666666666667) and guarded successes = 300/300 (1.0). The paired contingency table (analysis/paired_contingency_table.csv) is n11 = 299 (both pass), n10 = 1 (guarded pass only), n01 = 0 (baseline pass only), n00 = 0 (both fail). The absolute paired difference (guarded - baseline) is 0.0033333333333332993. The preregistered exact McNemar two-sided test returned p = 1.0, reflecting the extremely small number of discordant pairs. The paired bootstrap percentile 95% confidence interval (bootstrap_seed = 1729, B = 10000) reported with the analysis executor is [0.00, 0.01], bounding the plausible absolute paired improvement under the sampled suite and design.

Per-stratum diagnostics. Stratum-wise archived totals (strata defined in the task manifest by difficulty metadata) reveal: low difficulty — baseline 100/100, guarded 100/100; medium — baseline 100/100, guarded 100/100; high — baseline 99/100, guarded 100/100. The single discordant pair originates from the high-difficulty stratum. These per-stratum counts are provided as exploratory diagnostics in the archived artifacts and confirm that the limited discordance was concentrated in the highest difficulty bin in this synthetic suite.

Repair diagnostic and episode-level trace. The lone discordant episode (the single n10) triggered the prerogative repair stage. Validator traces for that episode attribute the initial flag to an omitted sequencing command required by the intent (a must_be_down_for_fields sequencing violation): the shared_initial candidate lacked the mandated 'admin down' step prior to a VLAN/MTU change. The constrained automated repair applied a single localized insertion of the missing sequencing command; deterministic_netops_validator_v5 subsequently reported valid == true for the repaired candidate. The repair provider trace and the validator trace for this episode are archived with the run; the manuscript reports this concise semantic summary rather than reproducing raw provider transcripts in the body.

Contamination and missingness. The contamination_summary.csv is empty (no contamination flags recorded), and the missingness summary (analysis/missingness_summary.csv) reports zero failed or unscorable episodes and zero excluded pairs under the preregistered exclusion rules. No pairs were removed for dual-call provider outages, and no single-call failures occurred that required sensitivity-analysis handling.

Statistical context and practical accounting. The combination of high baseline single-shot success (299/300) and a bounded repair budget (<=1 edit per flagged candidate) produced only a single deterministic rescue. Provider-call accounting shows that achieving that rescue required one additional model-stage call beyond the 300 shared_initial generations (total model_calls_used = 301), indicating the marginal model-call cost of the constrained repair policy in this run. All analysis artifacts (contingency table, condition summary, bootstrap parameters, and reconciliation reports) are archived for reproducibility and forensic inspection.

V. Discussion

Estimand conditioning and operational alignment. Conditioning on a single LLM draw per intent (shared_initial) corresponds to operational policies that constrain model calls for cost, latency, or governance reasons. Under such policies the guarded pipeline measures only deterministic validator and bounded repair rescue applied to the same candidate. Workflows that permit resampling, verifier-guided generation, or interactive steering change the estimand and typically increase achievable success rates by exploiting generator variance; those workflows must be evaluated under a different preregistered estimand.

Ceiling effects and the omitted pilot. The preregistration targeted detection of a 10 percentage-point absolute improvement with planned power ≈0.8 based on a pilot to estimate discordance. Because the optional pilot (n = 60) intended to estimate discordance was not executed, the main run confronted a near-ceiling baseline (299/300), producing minimal discordance (m = 1) and reduced power to detect small absolute differences. This practical outcome highlights the utility of small calibration pilots when generator performance may approach ceiling on chosen task suites.

Validator coverage and failure-mode dependence. The observed single failure mode was a sequencing omission; other classes (reachability violations, forbidden ACL removal) were not observed at scale on this synthetic suite. The marginal utility of a guarded pipeline therefore depends on (a) the deployment distribution of failure modes and (b) validator expressiveness: richer validators that detect reachability or ACL semantics may increase observed rescue rates. Conversely, if operator risk policies disallow automated edits, the guarded pipeline's deterministic rescue becomes operationally limited to detection and human escalation.

Repair budget and bounded edits. We intentionally limited automated repair to a single constrained edit to measure bounded deterministic rescue. That design measured only localized sequencing corrections; iterative repair, larger edit operators, or interactive verifier-guided generation would increase rescue opportunities but address a different estimand. The archived single repair trace enables forensic analysis of edit efficacy and could inform permitted edit-operator design in production pipelines.

Practical implications for NetOps CI/CD. Organizations must align evaluation estimands with operational choices: if pipelines permit multiple model calls or verifier-guided regeneration, evaluations should allow resampling and interactive steering to measure the achievable success rates under those policies. If model calls are tightly budgeted, the shared_initial estimand provides an operationally meaningful upper bound for deterministic verifier/repair rescue applied to a single candidate. Our artifact audit (model_calls_used = 301, repair_stage_count = 1) shows that in this run bounded repair produced measurable rescue at very low additional model cost for the single rescued episode.

VI. Limitations

- Synthetic benchmark: tasks and topologies were generated by the preregistered deterministic task generator; real-world heterogeneity may expose different failure modes and increase verifier marginal utility.
- Single model and validator: only gpt-5-mini and deterministic_netops_validator_v5 were evaluated; results may not generalize to other LLMs or richer/diverse validators.
- Ceiling effect: baseline single-shot performance was near 100% on this suite (299/300), producing limited discordance and reduced power to detect small absolute improvements relative to larger pre-specified targets.
- Pilot not executed: the preregistered pilot intended for discordance estimation and power calibration was not run; no post-preregistration power recalculation was performed.
- Estimand conditioning: the shared_initial protocol conditions the estimand on a single LLM candidate and therefore does not measure verifier benefit that would arise from allowing independent alternative LLM candidates per condition (resampling or iterative generation).
- Repair budget: the guarded condition permitted at most one automated repair per pair; larger or iterative repair strategies may yield different outcomes.

VII. Conclusion

We executed a preregistered, artifact-archived paired comparison between a verifier-assisted pipeline (deterministic validator + <=1 automated repair) and accepting a single-shot LLM candidate under the shared_initial estimand on 300 synthetic NetOps intents using gpt-5-mini and deterministic_netops_validator_v5. Baseline single-shot generation produced 299/300 unflagged verifier passes and the guarded pipeline 300/300, yielding an absolute paired improvement of 0.00333 (exact McNemar two-sided p = 1.0; 95% bootstrap CI [0.00,0.01]). Deterministic validator+repair rescued a single sequencing omission under the bounded repair budget. The near-ceiling baseline limits inferential sensitivity to small absolute improvements under the shared_initial estimand. We release full preregistration, execution, and analysis artifacts to support reproducibility and targeted follow-up studies that vary estimand conditioning, model strength, task distribution, or repair budget.

Disclosure Statement

This experiment and manuscript were produced by an autonomous CNSM Agentic-AI research run executed under the archived initial master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Human role: the human Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved pre-lock infrastructure; all literature search after the autonomy lock, autonomous study selection, preregistration (study id cnsmp2026-llm-verifier-predeployment-001), execution, analysis, manuscript drafting, AI peer review, and final compilation were performed autonomously under the locked master prompt. Models and orchestration: the declared generation model used was gpt-5-mini (execution/model_configuration.json declares requested_model = "gpt-5-mini"); adapter/orchestration framework: hosted_netops_gvr_v1. Validator and tooling: deterministic_netops_validator_v5 performed semantic and syntactic validation according to the preregistered full_intent_constraint_validation rule; the guarded condition permitted at most one automated repair call per pair implemented as a constrained edit operator. Preregistration: the confirmatory design, estimand, analysis executor, exclusion/missingness rules, and multiplicity plan were frozen prior to the main run and archived with the study id (preregistration SHA-256 = 425cb1652354120c f48f686b964723d64c76d0a8abe53cd5b5ad0eaa1af71fc7). Execution summary (archived): planned_episode_count = 600, completed_episode_count = 600, complete_pair_count = 300, model_calls_used = 301; provider audit: hosted_model_call_event_count = 301, stage_counts show shared_initial_generation = 300 and repair = 1. Pilot: the preregistration described an optional pilot (n = 60) for discordance estimation and power calibration; that pilot was not executed and no post-preregistration recalculation of required sample size was performed. Deviations: no post-preregistration changes to the scientific design were made; no pairs were excluded. Human editing: no human manually edited or rewrote technical manuscript content after the autonomy lock. Artifact availability: all raw outputs, logs, task manifests, validator traces, repair provider traces, and analysis artifacts referenced in this manuscript are archived in the run bundle associated with the study id. The archived run bundle contains the low-level repair provider trace documenting the single automated repair and subsequent validation pass; this manuscript reports a concise semantic summary of that repair in Results rather than reproducing full verbatim provider transcripts in the body.

REFERENCES

[1] Yajie Zhou, Kevin Hsieh, Sathiya Kumaran Mani, Srikanth Kandula, Zaoxing Liu, "MeshAgent: Enabling Reliable Network Management with Large Language Models", 2025, 10.1145/3771567

[2] Xiaolong Wei, Yuehu Dong, Xingliang Wang, Xingyu Zhang, Zhejun Zhao, Dongdong Shen, Long Xia, Dawei Yin, "Beyond ReAct: A Planner-Centric Framework for Complex Tool-Augmented LLM Reasoning", 2026, 10.1609/aaai.v40i40.40676

[3] Xu Liu, Peng Zhang, Anubhavnidhi Abhashkumar, Jiawei Chen, Weirong Jiang, "Automatic Configuration Repair", 2024, 10.1145/3696348.3696895

[4] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447

[5] X X Zhang, Xianming Gao, Peilin Tao, Tao Feng, "GraphSynth: Synthesis of Network Configuration Templates Using Large Language Models", 2025, 10.1145/3728725.3728742

[6] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[7] Yunze Wei, Wei, Kaiwen, Shaohui Du, Wang, Jianyu, Liu, Zhangzhong, Yawen Wang, Zhenxin Li, Congcong Miao, Xiaohui Xie, Yong Cui, "Automated Network Protocol Testing with LLM Agents", 2025, 10.48550/arxiv.2510.13248

[8] Fuliang Li, Haozhi Lang, Jiajie Zhang, Jiaxing Shen, Xingwei Wang, "PreConfig: A Pretrained Model for Automating Network Configuration", 2024, 10.48550/arxiv.2403.09369